

ECOM 4003 | Assembly Language Lab

Lab#0

Lab policy and Tools

Dr. Ruba R. Salamah

Eng. Raghad A. Hilles &

Eng. Yasser O. Hejazi

Grading policy

10%

Lab Quizzes

20%

Homework labs

30%

Total

Lab Objective

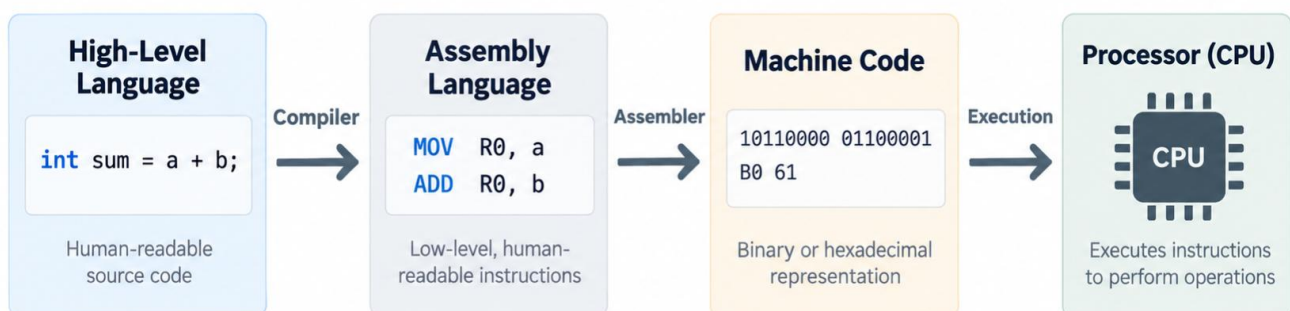
- To be familiar with Visual studio assembler and debugging tools.
- To be familiar with Assembly Language.

What is Assembly Language?

- High-level languages such as C++ let us express an idea in a few familiar statements. Assembly shows more of the individual steps the processor performs. For example, adding two values may require loading a value into a register and then adding another value to it.
- Assembly Language is therefore a readable way to work close to the processor. It is not the same as machine language: a program still needs to be assembled before the CPU can execute it.

From High-Level Language to the Processor

How code moves from human-readable form to execution



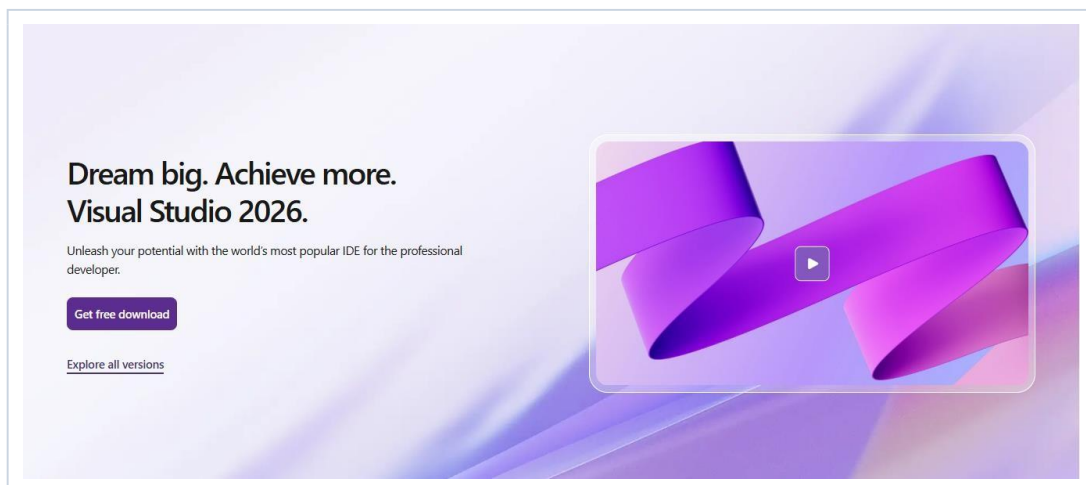
Assembly language helps us understand how software communicates directly with hardware.

The Tools and How They Work Together

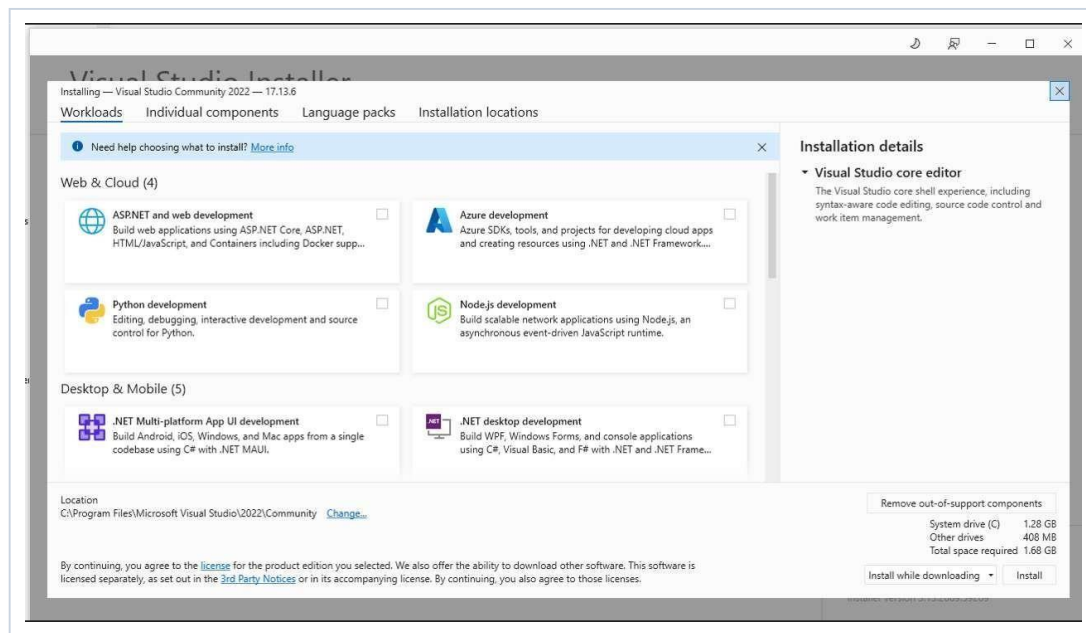
- **Visual Studio:** It is the development environment (**IDE**). It gives us a place to organize a project, write source code, build the program, run it, and debug it. Visual Studio is not the Assembly language and it is not the assembler.
- **MASM and the assembler:** MASM means Microsoft Macro Assembler. The assembler translates our .asm source file into an object file containing machine code. In the 32-bit workflow used in this course, the assembler program is commonly named ml.exe.
- **The linker:** The object file is not always a complete program. The linker combines object files with any required libraries and creates the executable program, usually an .exe file.
- **The debugger:** It lets us pause a running program and examine it one instruction at a time. We can inspect registers and memory, and later use flags to understand decisions made by the processor. This helps us see why a program behaves as it does and find mistakes.

Get started

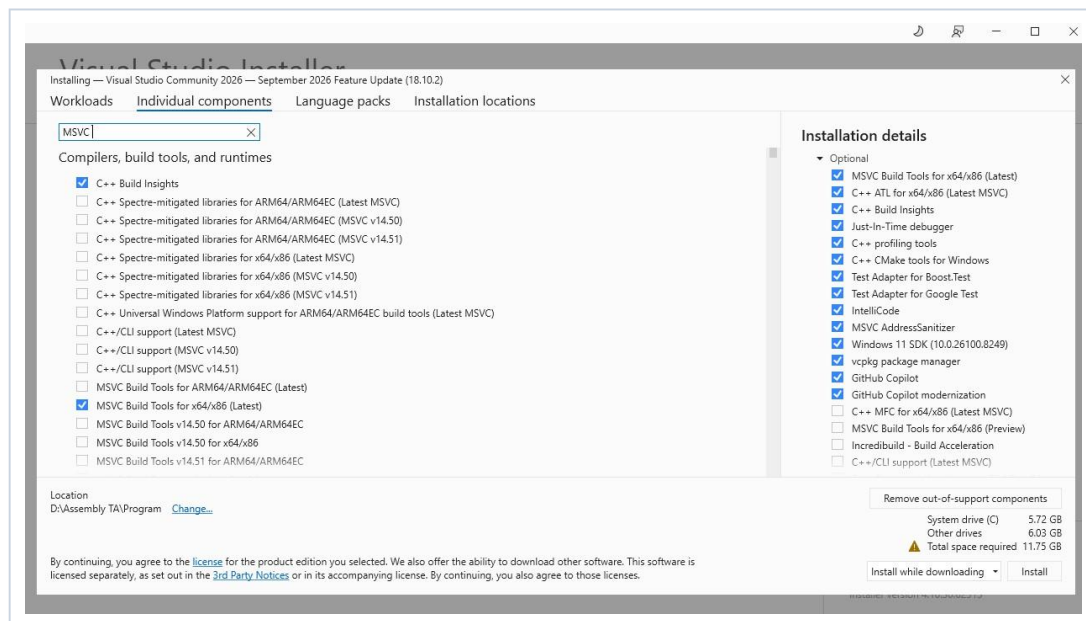
- Go to the website <https://visualstudio.microsoft.com/> in your web browser.
- Click on download visual studio



- After download open installer

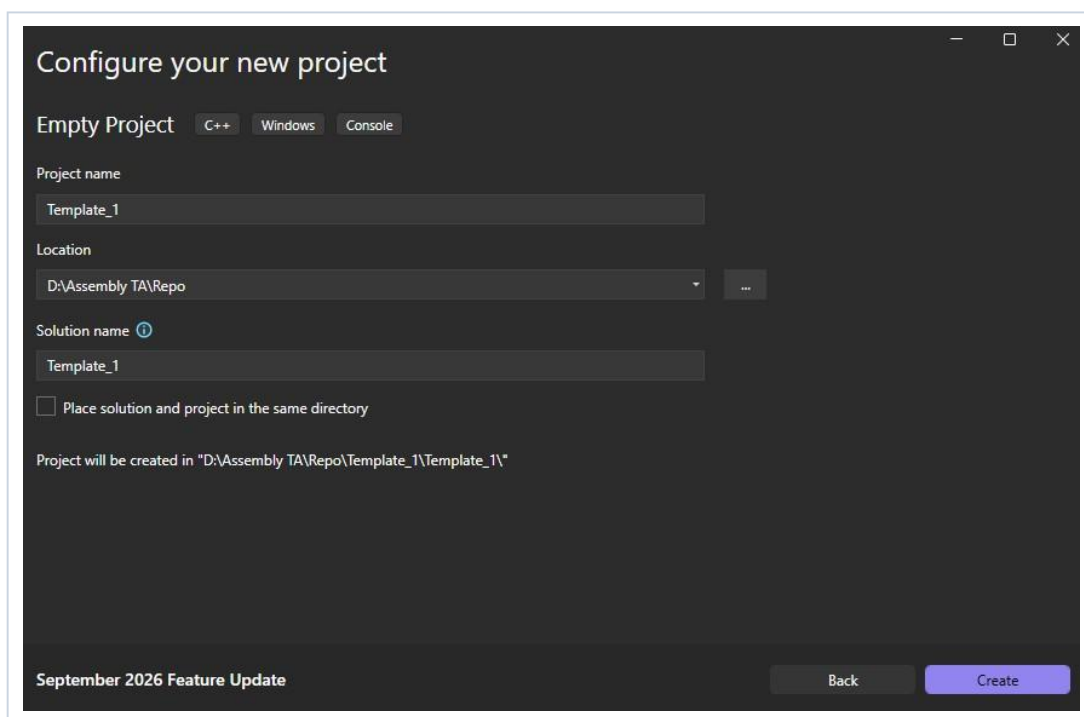
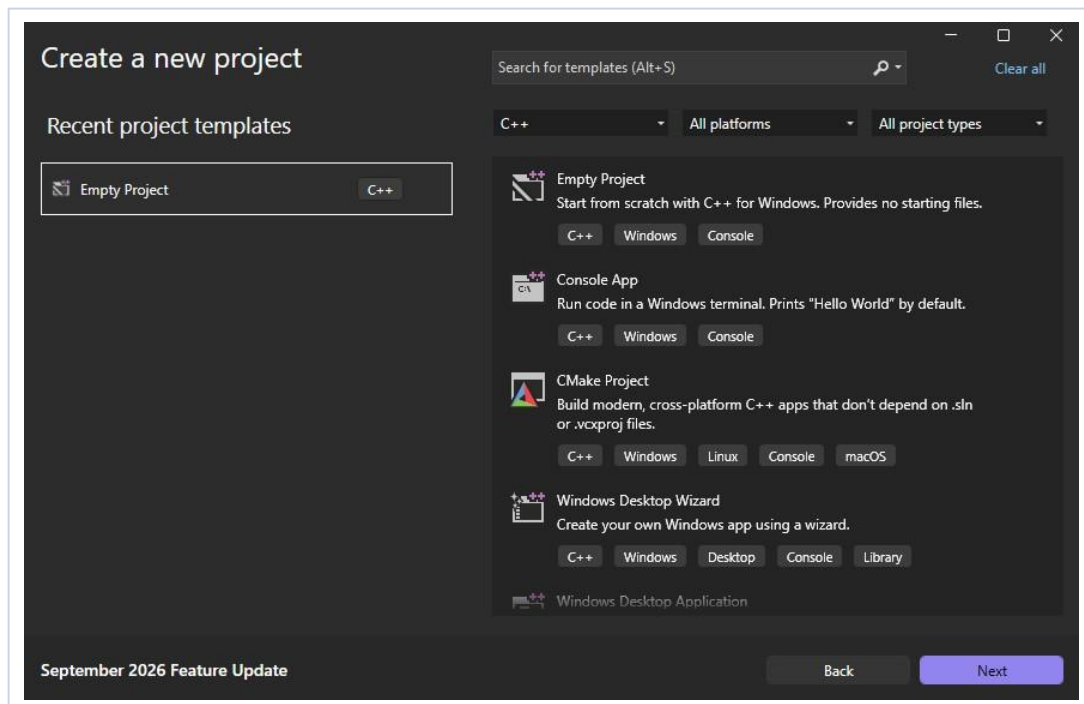


- You should select the workloads before start development.
- First Desktop development C++ should select but before it you should install components for it.
- From individual components check all components needed.
- Search for component and check it for installing.

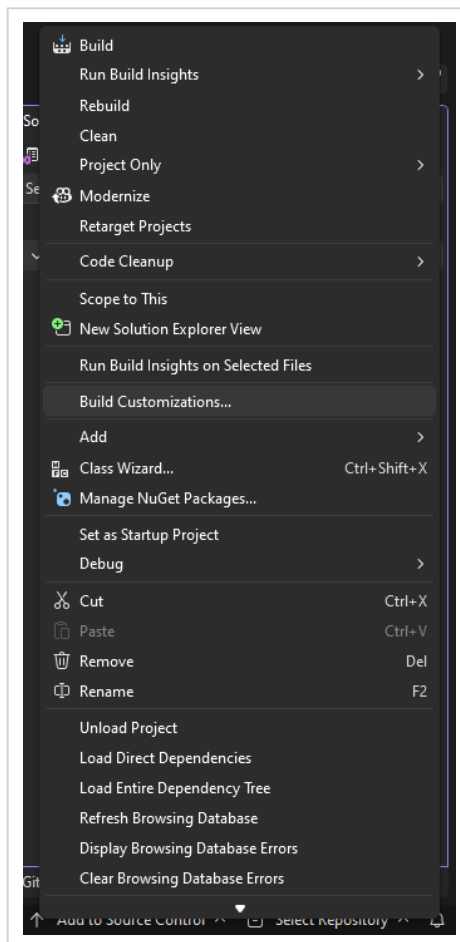
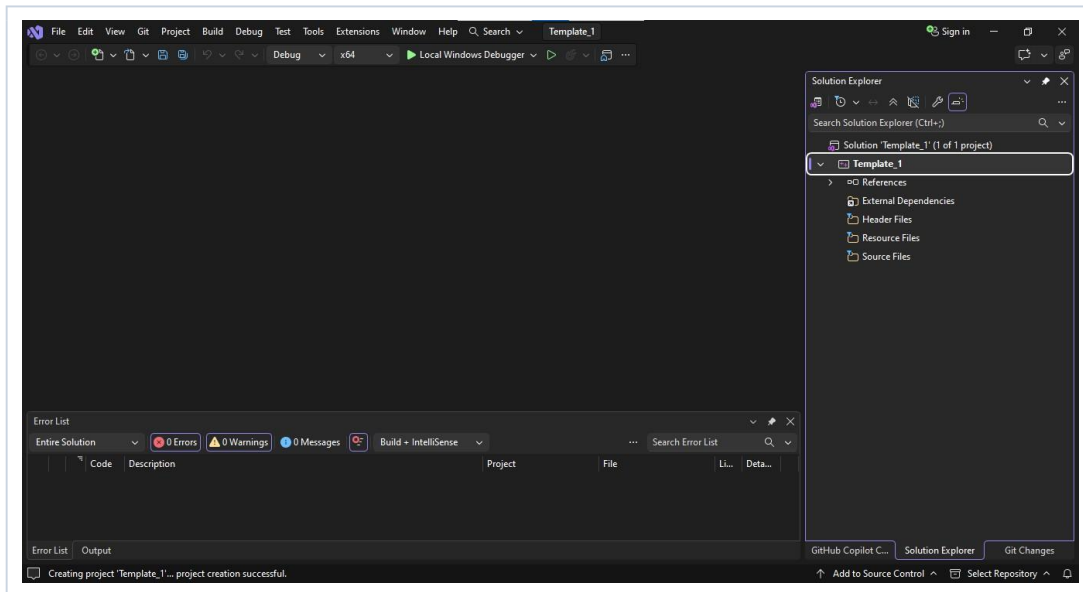


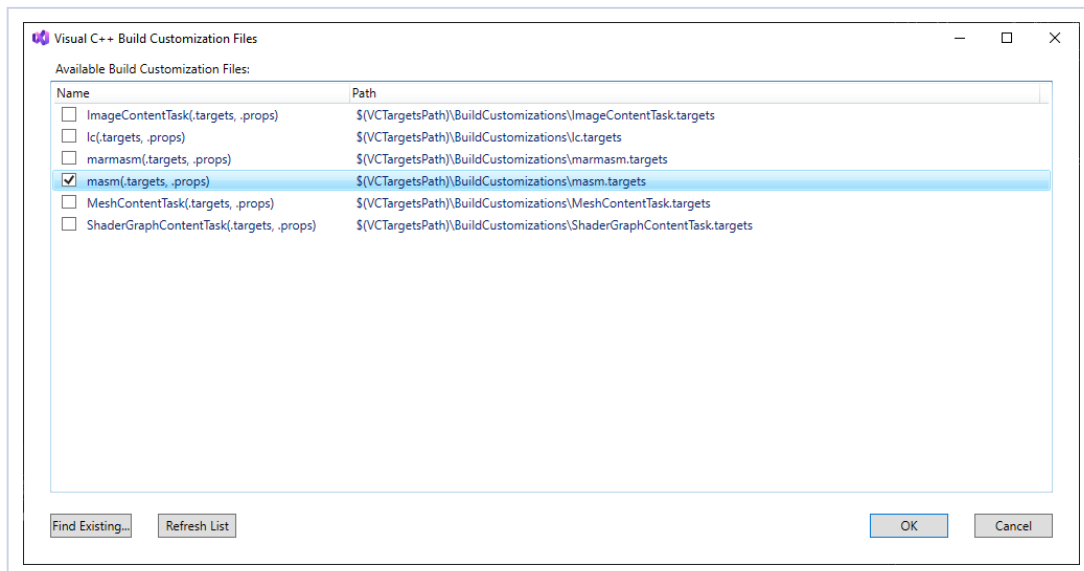
- After select all components check desktop development with C++.
- Components as you see on the right side.
- Setting up MASM in Visual Studio. Create a new project. After installing Visual Studio, open Visual Studio and create a new project, and from there select the Empty Project. After selecting an empty

project, it will ask you for a project name. You may name it whatever you would like. Then click "Create".

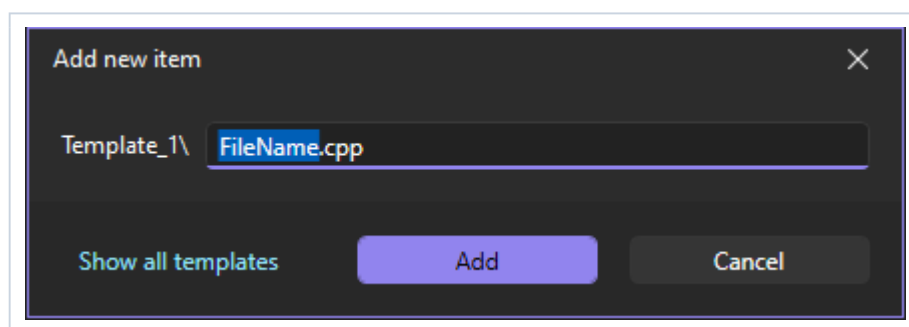
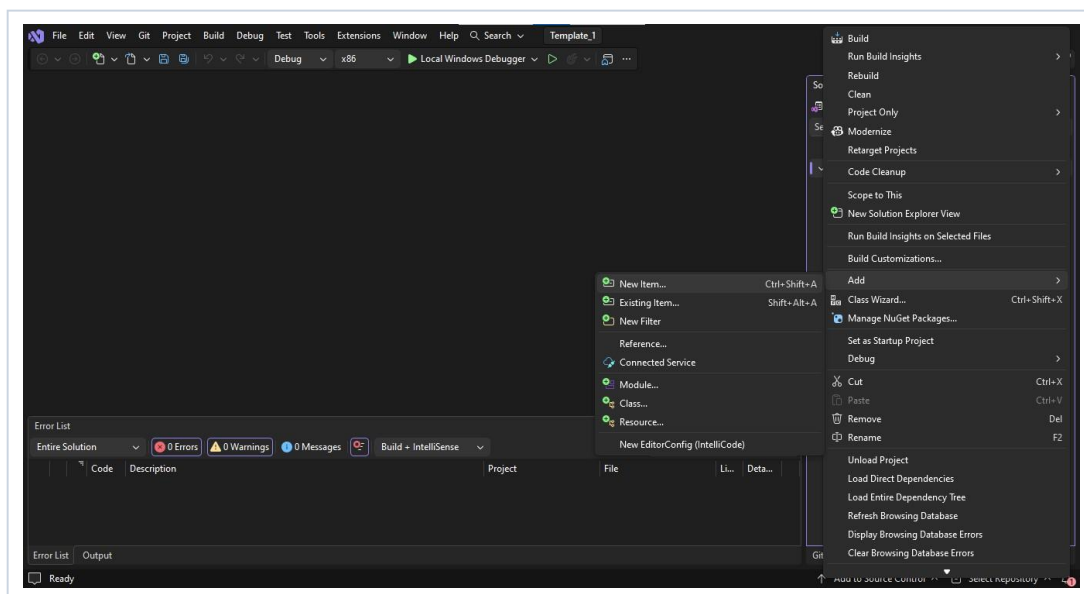


- After you have created the project, go to the solution explorer on the left side and right-click on your project name, from the menu navigate to "Build Dependencies -> Build Customizations". A small window will pop up from the window, check MASM, and hit "Ok".

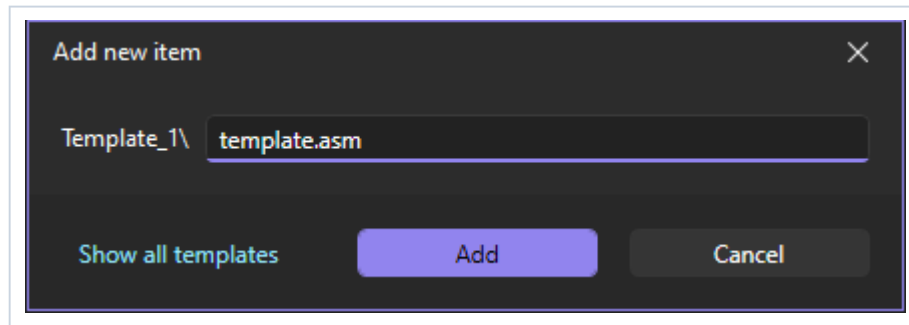




- Now right-click on Template (project name) and navigate to Add -> New File.

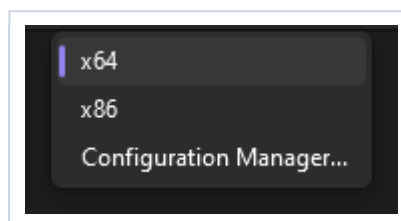
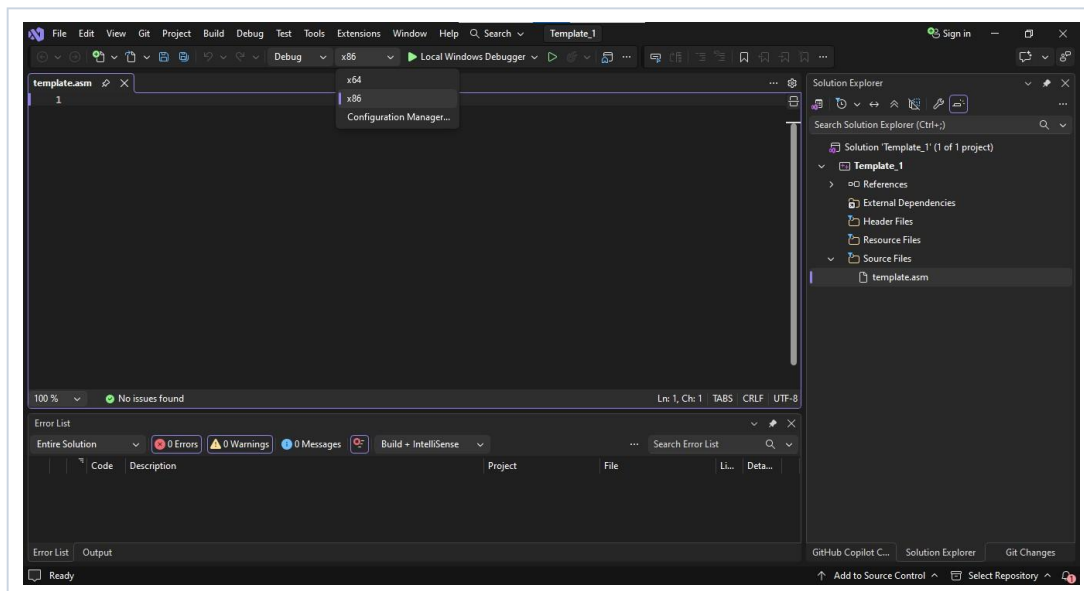


Since we are setting up an environment for **MASM (Assembly)** in Visual Studio, not C++ we must change the .cpp to .asm then click on add.



- You are advised to check that the build mode is set to x86 and not x64.

Note / The Microsoft Macro Assembler (MASM) is an x86 assembler that uses the Intel syntax for MS-DOS and Microsoft Windows.



Creating A Project Template for a Visual Studio Project:

Write this simple code :

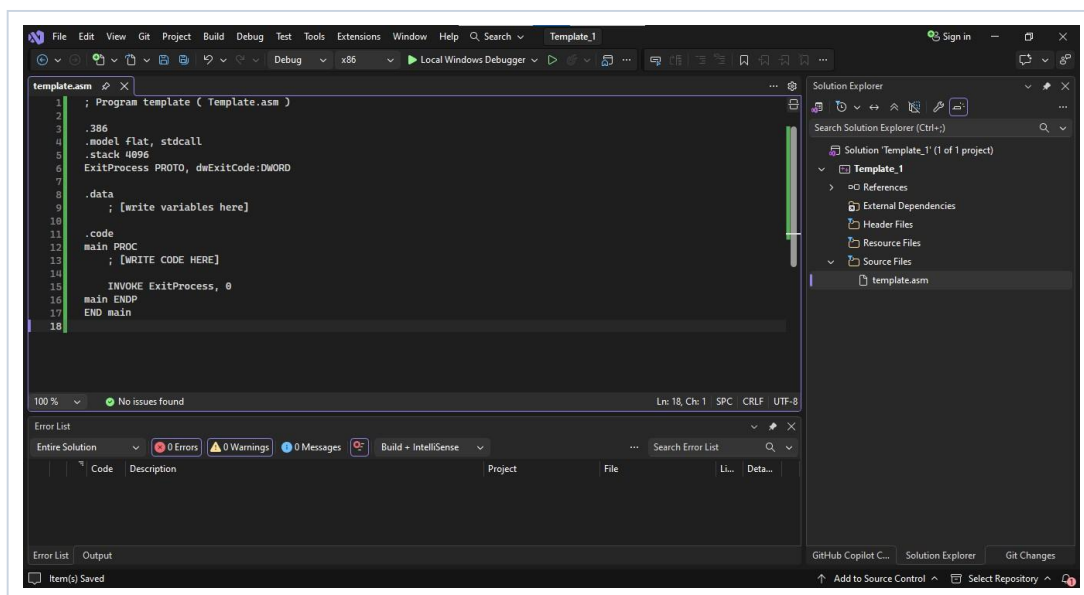
```
; Program template ( Template.asm )

.386
.model flat, stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD

.data
; [write variables here]

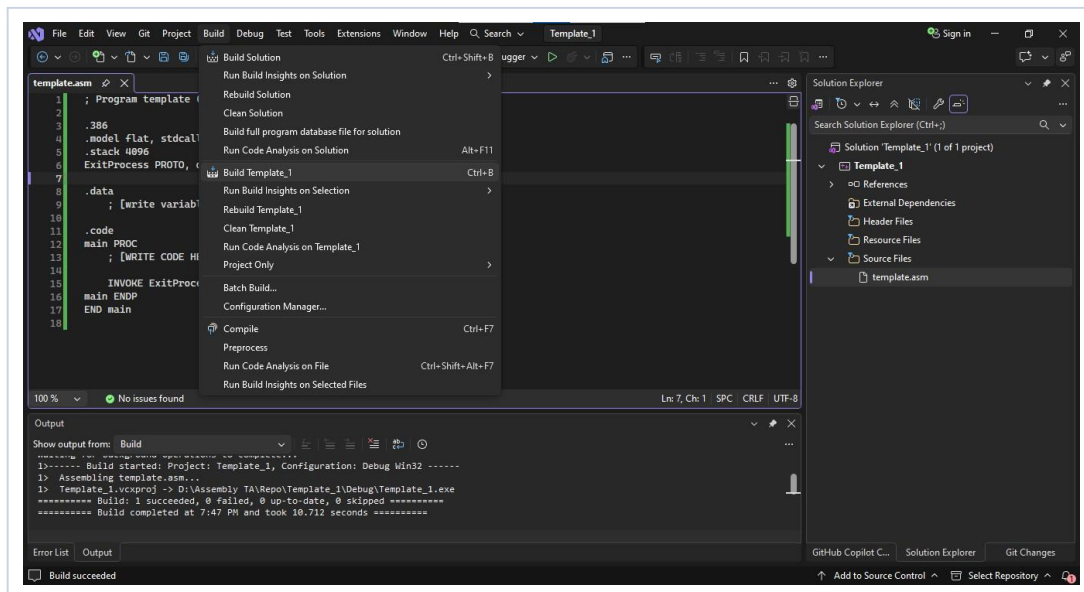
.code
main PROC
; [WRITE CODE HERE]

INVOKE ExitProcess, 0
main ENDP
END main
```



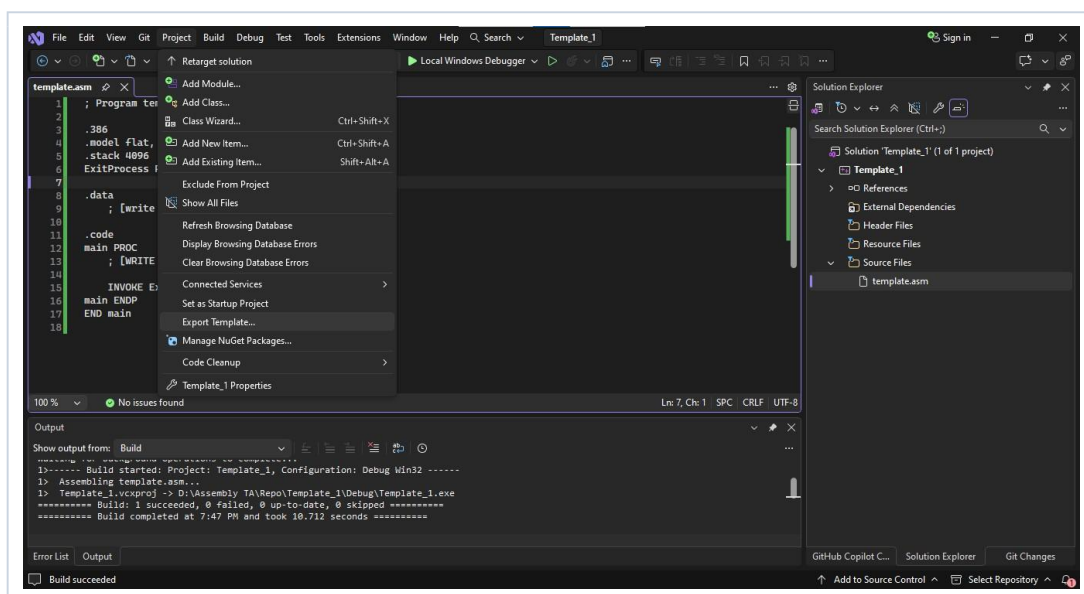
- Now to test your code go to Build -> Build Template_1 then it will give you :

Build succeeded.



- Final step : Exporting the template

Go to Project on the top, and then from the drop-down menu, select "Export Template" A window will pop and then hit next. it will then ask you for a template name, icon, and description. Name it "Assembly Language Project" and hit "Finish".



Export Template Wizard

 **Choose Template Type**

This wizard will allow you to export a project or project item from the current solution to a template which future projects can then be based upon.

Which type of template would you like to create?

☒ Project template

A project template will allow a user to create a new project based on your exported project. A user will be able to utilize your template from the New Project dialog box for client projects and from the New Website dialog box for websites.

☐ Item template

An item template will allow a user to add your item to one of their existing projects. Your template will be available to the user from the Add New Item dialog box.

From which project would you like to create a template?

Template_1

< Previous **Next >** Finish Cancel

Export Template Wizard

 **Select Template Options**

Template name:

Template_1

Template description:

<No description available>

Icon Image:

 Browse...

Preview Image:

 Browse...

Output location:

C:\Users\Hp\OneDrive\Documents\Visual Studio 18\My Exported Templates\Template_1.zip

☒ Automatically import the template into Visual Studio

☐ Display an explorer window on the output files folder

< Previous Next > **Finish** Cancel